CareerSetu: A Deterministic and Explainable AI Framework for Resume Intelligence, Skill-Gap Analysis, and Internship Hiring

Kirti S. Vispute
Department of Computer Engineering
Vidyalankar Institute of Technology, Mumbai, India
Email: kirtisvispute2@gmail.com

Abstract—Internship hiring is a two-sided information problem: students rarely learn *why* a resume was rejected, and recruiters are forced to screen large volumes of unstructured documents by hand. Existing applicant-tracking and job-recommendation tools generally resolve this by substituting one opaque process (manual screening) for another (an uninspectable machine-learning score), which does not restore trust to either side of the market. This paper presents CareerSetu, a full-lifecycle internship-hiring platform built around a contrasting design commitment: the resume score is computed by a deterministic, weighted rule engine rather than a language model, so that every point gained or lost can be traced back to a named dimension, a fixed weight, and a human-readable reason. CareerSetu scores a resume against the specific posting a student targets rather than a generic ideal, performs a three-pass evidence-weighted skill-gap analysis, converts the gap into a phased learning roadmap, and gives recruiters a configurable, per-role weighted ranking of applicants. Large language models are retained only as an optional layer that renders structured facts into prose explanations, behind a three-stage grounding-primary-validation pipeline that cross-checks independent estimates of the same quantity before reporting a confidence level. The system is implemented as a modular monolith (Next.js, Node.js/Express, MongoDB) spanning eighteen backend modules, is engineered to run entirely on free-tier infrastructure for roughly five hundred concurrent users, and layers role-based access control, JWT-based authentication, and OWASP-aligned input validation beneath the hiring workflow. A worked case study and a structured self-assessment across security, performance, and production-readiness criteria are used to evaluate the design, and the paper closes by situating CareerSetu's determinism-first approach against the black-box scoring and matching literature it departs from.

Index Terms—Applicant Tracking System, Explainable Artificial Intelligence, Resume Parsing, Skill-Gap Analysis, Internship Recommendation, Modular Monolith, Candidate Ranking.

I. INTRODUCTION

The internship market in India operates at a scale that neither side of it is tooled to handle. A single posting from a mid-sized company routinely draws hundreds of applications, almost all of them unstructured PDF or DOCX documents that differ in layout, section ordering, and terminology. On the recruiting side, this volume forces a choice between slow manual screening and a blunt automated filter, and in practice the two blend into a screening process that is both slow and inconsistent between reviewers. On the student side, the experience is the mirror image of that opacity: a resume is submitted, a rejection follows, and nothing in between explains which specific requirement was unmet or which change would have mattered. The student repeats the same mistakes across dozens of applications because the system that rejected them never told them what to fix.

The pattern that connects both halves of the problem is that the resume is the interface between the student and the company, and that interface is opaque from both directions. The student cannot see how a parser or a scoring model reads their document; the recruiter cannot see, beyond a single aggregate number, how a candidate compares to the actual requirements of the role. A platform that wants to fix internship hiring, as opposed to merely digitizing it, has to make that interface legible to both sides at once, and it has to do so without pretending that an opaque scoring model is somehow fairer than opaque human judgment.

A large share of existing applicant-tracking and resume-screening literature and tooling addresses one half of this problem at a time. Resume-parsing and NLP-for-HR research has concentrated on extraction accuracy and semantic matching quality, generally trusting a trained model to produce a usable score [1, 2]. Job- and candidate-recommendation systems concentrate on ranking quality under embedding- or similarity-based matching [3, 4, 5]. Fairness and explainability work in AI-driven recruitment is a comparatively recent and still-developing strand, motivated precisely by the observation that an accurate model is not the same thing as an accountable one [6]. What is less common is a system that commits, architecturally, to *not* letting a model own the number that decides a student's outcome, while still integrating the parsing, matching, and fairness concerns that the literature treats separately.

This paper presents CareerSetu, an internship-hiring platform designed around that commitment. Its central engineering

position is that the resume score is produced by a deterministic, auditable rule engine, and that language models are used only where their non-determinism is harmless — principally, for turning already-computed facts into readable prose. The contributions of this work are as follows.

1. A seven-dimension, weighted, context-aware resume-scoring engine that scores a resume against the *specific* posting a student selects (or, absent one, a role taxonomy) rather than a generic rubric, and that never penalizes a student for a requirement the posting itself does not state.
2. A three-layer AI orchestration pipeline (grounding, primary computation, cross-validation) that reports its own confidence and explicitly flags disagreement between independent estimates of the same quantity, instead of returning a single number with false precision.
3. A three-pass, evidence-weighted skill-gap engine that distinguishes a skill merely listed from one demonstrated through a project or a certification, and that converts the resulting gap into a phased, actionable learning roadmap.
4. A configurable, per-role candidate-ranking model for recruiters, in which hiring weights are a business parameter rather than a hard-coded constant, together with a complete applicant-tracking pipeline, scheduled WebRTC interviews, and real-time messaging.
5. A production-shaped, modular-monolith implementation engineered under an explicit zero-infrastructure-cost constraint, with a documented graceful-degradation path for every optional external dependency.

The remainder of this paper is organized as follows. Section II surveys related work in resume parsing, job recommendation, and fair/explainable hiring, and positions CareerSetu against it. Section III states the problem and the design objectives more formally. Section IV describes the system architecture. Section V is the methodological core of the paper and details the AI subsystem and the resume-scoring engine. Section VI covers implementation — the technology stack, data model, and security and performance engineering. Section VII presents a worked case study and a structured self-assessment of the system. Section VIII discusses limitations and future work, and Section IX concludes.

II. RELATED WORK

A. Resume Parsing and NLP for Human Resources

Converting an unstructured resume into structured, comparable fields is a prerequisite for any downstream scoring or matching step, and it remains a non-trivial extraction problem because of inconsistent section headings, multi-column layouts, and mixed formatting conventions. Otani et al. survey the broader application of natural-language-processing techniques across the human-resources pipeline, from parsing through matching to generation, and note that extraction quality is frequently the limiting factor for everything built on top of it [1]. Amir Hussin et al. describe an NLP-based resume-screening system that couples extraction with ATS-oriented resume optimization, reporting on the practical gap between what a resume contains and what an automated screener actually reads [2]. CareerSetu's parsing layer follows the same extraction-first philosophy but treats the *reliability* of the extraction as a first-class, propagated signal: a parse-quality score, rather than being discarded once sections are identified, is later multiplied into the final resume score whenever confidence falls below a fixed threshold, so that a poorly parsed resume cannot silently produce an inflated, falsely precise result.

B. Semantic Matching and Job Recommendation

A second strand of work targets the matching problem directly: given a candidate's profile and a set of postings, rank the postings (or candidates) by fit. Jain et al. propose a semantic-analysis-based job-recommendation system that embeds resume content and job descriptions into a shared space for similarity ranking [3]. Schlippe and Bothmer's Skill Scanner targets a three-sided market — employers, job seekers, and educational institutions — and recommends skills and courses based on gap analysis against labor-market demand [4]. Imankulova et al. provide a broad review of career-recommendation approaches and the challenges that recur across them, including cold-start candidates, skill-taxonomy drift, and the difficulty of validating recommendation quality without longitudinal outcome data [5]. These systems optimize primarily for ranking quality; CareerSetu adopts a similar three-pass matching idea for its skill-gap engine (Section V-E) but subordinates the matching score to an explicit evidence hierarchy — a skill demonstrated in a project outranks the same skill merely listed — and reports the two cases differently to the student rather than collapsing them into one percentage.

C. Fairness and Explainability in AI-Driven Hiring

Because hiring decisions carry real consequences, a growing body of work examines bias and the absence of explainability in automated recruitment pipelines. Assafuah et al. propose a multi-stage framework for detecting and mitigating bias in AI-driven recruitment systems, arguing that post-hoc fairness auditing is necessary precisely because the underlying scoring models are themselves opaque [6]. CareerSetu approaches the same concern from the architecture side rather than the audit side: by keeping the scoring function deterministic and weight-based, there is no learned model whose bias needs to be audited after the fact, and every deduction is reconstructible from a fixed, inspectable weight table. This does not eliminate the possibility of an unfair *weighting* decision, but it does eliminate an entire class of failure — an inconsistent or drifting score for the same input — that an audit of a black-box model would otherwise have to detect empirically.

D. Internship and Placement Management Systems

A separate, more operational literature addresses internship and placement management as a workflow-digitization problem. Abdullah et al. describe a web-based internship-management

Table 1: Stakeholder Pain Points in Internship Hiring

Stakeholder	Observed pain	Consequence
Student	No feedback loop on rejection	Repeats the same mistakes across applications
Student	Cannot see the gap between held and required skills	No concrete direction for learning
Recruiter	Hundreds of unstructured documents per posting	Manual shortlisting, high time-to-hire
Recruiter	Screening varies by reviewer	Unfair, unrepeatable decisions
Both	Scheduling and feedback live in email/chat	Lost context, no audit trail

system focused on replacing paper-based placement-cell processes with a tracked online workflow [7]. Systems in this category typically digitize scheduling, document submission, and status tracking without attempting resume intelligence at all. CareerSetu occupies the space between the two literatures surveyed above and this one: it carries a full applicant-tracking and interview-scheduling workflow, as in placement-management systems, but grounds every stage of that workflow in the same explainable scoring and matching engine used to advise the student, rather than treating hiring-pipeline tracking and candidate intelligence as separate concerns.

E. Architectural and Security Grounding

Two implementation-level decisions in CareerSetu are informed directly by the software-engineering literature rather than the HR-tech literature. Al-Debagy and Martinek's comparative review of microservice and monolithic architectures documents the operational-complexity and deployment-cost tradeoffs that motivate choosing one style over the other for a given team size and budget [8]; this comparison underlies the architectural choice discussed in Section IV-A. Pratama et al.'s security assessment of JWKS-based JWT authentication enumerates concrete attack vectors against token-based authentication schemes and the mitigations available to each [9]; the authentication design discussed in Section VI-C is evaluated against the same vector set.

III. SYSTEM OVERVIEW AND DESIGN OBJECTIVES

A. Problem Statement

Table 1 restates the stakeholder-level failures that motivate this work. The recurring theme is that neither side of the market can see what the other side sees: the student cannot see how a machine reads their resume, and the recruiter cannot see how a candidate compares to the role's actual requirements beyond a single ranked list.

B. Design Objectives

CareerSetu was designed to satisfy six objectives simultaneously: (1) convert an unstructured resume into structured, machine-comparable data; (2) score that data against the specific internship a student targets, rather than a generic ideal, and explain every point lost; (3) turn the resulting gap into a concrete, phased learning roadmap; (4) give recruiters a configurable, auditable, weighted ranking of applicants; (5) run the full hiring lifecycle — application, screening, interview, and decision — inside one system rather than across email and spreadsheets; and (6) do all of the above at effectively zero infrastructure cost, so the platform remains viable for an unfunded student-run deployment serving on the order of five hundred concurrent users. The sixth objective is not a scope reduction applied after the fact; it is a constraint that was active from the first architectural decision and is traceable through nearly every choice described in Sections IV through VI, from the selection of a single deployable backend process to the requirement that every external dependency degrade gracefully rather than fail hard.

Table 2: Target Users and Primary Tasks

Role	Primary jobs-to-be-done
Student / Intern	Upload resume, understand score and deductions, close skill gaps, discover and apply to internships, attend interviews, track application status
Company (owner / CEO / HR)	Post internships, review a ranked applicant list, manage a hiring pipeline, schedule and conduct interviews, review analytics
Admin (verification / moderation / support)	Verify company accounts, moderate abuse reports, review submitted certifications, audit platform activity

C. Target Users

Table 2 lists the three top-level roles the platform serves. The role model is intentionally finer-grained than three entries suggest: a company account, for instance, may be an owner, a CEO-level account, or an HR sub-account, each with different permissions over the same company's data, and the admin role is split into verification, moderation, and support specializations so that no single account can both approve a company and moderate a report against it.

IV. SYSTEM ARCHITECTURE

A. Architectural Style

CareerSetu is implemented as a *modular monolith with a shared kernel*: a single deployable Node.js process, organized into eighteen feature modules, each owning its own routes, controllers, services, models, and validators, plus a shared kernel that holds only cross-cutting infrastructure (authentication, caching, logging, and the AI-provider router). This is a deliberate departure from a microservices decomposition. Al-Debagy and Martinek's comparison of the two styles identifies deployment cost, operational complexity, and team size as the dominant factors in choosing between them [8]; under a zero-cost, single-developer-team constraint, each of those factors favors a monolith (Table 3). The design nonetheless keeps module boundaries strict — a module may not import another module's

Table 3: Monolith vs. Microservices Under CareerSetu's Constraints

Criterion	Microservices	Modular monolith (chosen)
Deployment cost	N services × hosting	One free-tier instance
Operational load	Service discovery, distributed tracing	One process, one log stream
Team size needed	Multiple teams	Single-developer team
Transaction scope	Distributed / saga	Single database session
Future migration	—	Module seams already present

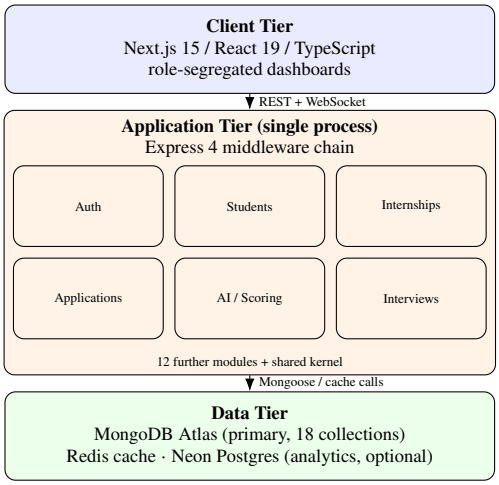


Figure 1: Three-tier logical architecture of CareerSetu.

internals — specifically so that the AI, notification, and reporting modules can later be extracted into independent services without a rewrite, should the scale objective in Section III-B ever be exceeded.

B. High-Level Architecture

Figure 1 shows the three logical tiers. The client tier is a Next.js application that renders role-segregated dashboards for students, companies, and administrators. The application tier is a single Express process that terminates every HTTP and WebSocket connection, routes requests through a deliberately ordered middleware chain (compression, security headers, CORS, body parsing, input sanitization, request-ID tagging, logging, and rate limiting, in that order), and dispatches into one of the eighteen feature modules. The data tier is intentionally polyglot in a narrow sense: MongoDB holds all transactional and resume-derived data, while an optional PostgreSQL instance holds only append-only analytics events, so that the two stores never need to be joined and the analytics path can be entirely absent without breaking any core feature.

C. Feature Modules and Communication

The eighteen backend modules are: authentication, students, companies, internships, applications, candidates, interviews, chat, calendar, notifications, reports, admin, ats-scoring, analytics, portfolio, skills, ai, and uploads. Table 4 summarizes how the client and server communicate across the lifecycle of a single interaction. REST over HTTPS carries ordinary CRUD and scoring traffic; Socket.io carries chat messages, presence, and interview-room signaling; and interview audio/video is carried peer-to-peer over WebRTC, so that media never transits the server — a choice that keeps live interviews viable on a free-tier deployment, since the server's only role in a call is relaying signaling messages rather than relaying media itself.

Table 4: Communication Channels

Channel	Protocol	Used for
REST	HTTPS / JSON	CRUD, scoring, reports, Bearer-JWT auth
WebSocket	Socket.io	Chat, presence, interview roster, WebRTC signaling
Peer-to-peer	WebRTC (SRTP/DTLS)	Interview audio, video, screen sharing
Server-to-user	SMTP	OTP codes, interview and application notices

V. METHODOLOGY: THE AI SUBSYSTEM

A. Design Philosophy: Deterministic-First AI

The organizing principle of CareerSetu's AI subsystem is that *scoring logic is deterministic and rule-based, and a language model is an optional enhancement layer that never owns a number*. This is stated first because it inverts the default assumption in much of the literature surveyed in Section II, where a trained model typically produces the headline score directly. Four requirements motivate the inversion. Reproducibility requires that the same resume, scored against the same posting, produce the same number on every run; a generative model does not guarantee this unless heavily constrained. Explainability requires that a deduction be traceable to a cause, not merely reported as a model output. Fairness and auditability require that the scoring weights be a frozen, inspectable table rather than millions of opaque parameters. Finally, cost and availability require that the core feature keep working, at zero marginal cost, even if every external AI provider is unreachable. Language models are therefore retained only where a wrong output is cheaply recoverable — principally, generating a fluent explanation sentence from already-computed facts — and never where a wrong output would silently become a student's score.

B. Three-Layer AI Architecture

AI-assisted features in CareerSetu are organized into three layers, mirrored on both the client (for instant feedback) and the server (as the authoritative path). The *grounding* layer collects only verifiably real signals — parsed resume fields, profile data, and posting requirements — and determines whether enough signal exists to proceed; if not, the system reports low confidence and an explicit "insufficient data" state rather than guessing. The *primary computation* layer runs the deterministic scoring and matching engines described in Section V-D; an LLM may optionally produce prose for the same already-computed

facts, but never substitutes for them. The *validation* layer cross-checks two or more independent estimates of the same quantity and converts their agreement or disagreement into a confidence score. Equation (1) gives the cross-validation rule used throughout the platform, where δ is the absolute difference between a primary and a secondary estimate of the same quantity and τ is a per-metric tolerance:

$$\text{confidence} = \begin{cases} b + 10, & \delta \le 3 \\ b - \delta, & 3 < \delta \le \tau \\ b - \min(35,\, 2\delta), & \delta > 2\tau \end{cases} \tag{1}$$

where b is a base confidence value. A resulting confidence at or above 85 is reported as the *high* tier, at or above 60 as *medium*, and below that as *low*, with an accompanying trace that states which two estimates were compared and by how much they disagreed. This design treats agreement between independent estimates as evidence and a single estimate alone as merely an assertion, and it allows the platform to report its own uncertainty rather than a falsely precise number.

C. Graceful Degradation of AI Providers

Every AI-assisted request passes through a provider ladder before any external call is attempted: if AI features are disabled, or a per-user rate limit is exceeded, or local heuristics are configured to run first (the production default), the local deterministic path executes immediately; only after that does the system attempt a primary LLM provider and then a fallback provider, with any failure falling through to the next rung and, in the worst case, back to the local heuristic. Responses are cached for twenty-four hours keyed by a hash of the request payload rather than the requesting user, so that two requests with identical inputs — regardless of who issued them — share a cache entry, and a changed resume produces a new hash and therefore a natural cache miss with no manual invalidation logic required.

D. The Resume Score Engine

The resume score engine is the deepest component of the system and the one most directly responsible for the explainability objective stated in Section I.

1) Scoring Formula

Given a resume and, optionally, a specific internship posting, the engine computes seven independent dimension scores $s_i \in [0, 100]$, each with a fixed weight w_i, and combines them as

$$\text{overall} = \text{round}\left(\sum_{i=1}^{7} w_i\, s_i\right) \times \begin{cases} q, & q < 0.75 \\ 1, & q \ge 0.75 \end{cases} \tag{2}$$

where q is a parse-quality confidence term described in Section V-D4, clamped afterward to the range $[0, 100]$. Table 5 lists the seven dimensions and their weights. Keyword matching and demonstrated technical skill dominate the total because those are what an automated screen and a recruiter's first pass actually key on; the *projects* weight is set higher than experience or education because, for a student audience, a project is typically the primary artifact of competence a candidate actually possesses; the *experience* weight is correspondingly reduced, and is governed by a fairness rule (Section V-D2) that never penalizes the absence of prior work experience unless the posting itself states a requirement for it.

Table 5: Resume Score Engine: Dimension Weights

#	Dimension	Weight	Points
1	Keyword Match	0.35	35
2	Technical Skills Match	0.15	15
3	Projects & Achievements	0.15	15
4	Experience Match	0.10	10
5	Education Match	0.10	10
6	Structure & Formatting	0.10	10
7	Certifications	0.05	5
Total		**1.00**	**100**

2) Per-Dimension Methodology

Keyword Match operates in one of three scoping modes, selected in priority order: against a specific posting's required and preferred skills if one is selected, against a role taxonomy's core and emerging skills if a target role is selected instead, and against the student's own declared skills only as a last resort. The required and preferred percentages are combined as $0.8\,p_{\text{req}} + 0.2\,p_{\text{pref}}$, and all comparisons are canonicalization-aware, so that `Node.js`, `Node`, and `NodeJS` are treated as satisfying the same requirement rather than being counted as three distinct, partially-matched tokens.

Technical Skills Match asks a different question than keyword presence: whether there is *evidence* that the candidate holds the skill, drawing on the same underlying matching computation used by the standalone skill-gap feature (Section V-E), so the two surfaces of the platform can never disagree with each other about the same candidate.

Projects & Achievements scores each project out of 100 from a base of 35 points, with additive credit for technology-stack breadth, a substantive description, and the presence of a repository or live-demo link, and then scales the result by a role-relevance multiplier $0.4 + 0.6r$, where r is the overlap ratio between the project's technologies and the target role's required skills. This multiplier is a deliberate compromise: an off-stack project still demonstrates initiative and retains 40% of its value, but it cannot inflate a role-specific score to the same degree as directly relevant work.

Experience Match encodes the fairness rule explicitly: when a posting states no experience requirement, or states a requirement of zero months (including phrases such as "fresher" or "entry-level"), the dimension scores 100 regardless of the candidate's actual experience, and a deduction is possible only when the posting states a non-zero requirement that the candidate genuinely does not meet.

Education Match is similarly evaluated only against requirements the posting states, and uses a synonym table so that

equivalent degree names (for example, `B.Tech`, `B.E`, and `Bachelor of Technology`) are treated as identical rather than being penalized for a superficial string mismatch.

Structure & Formatting combines section coverage, an independent rule-based formatting score, a readability estimate, and a length-fit term that rewards a resume in a target band of roughly 280 to 950 words.

Certifications is a bonus-only, relevance-filtered dimension: certifications unrelated to the target role are still shown to the student but contribute little to the score, so that a large number of unrelated badges cannot outrank one certification that is directly relevant.

3) Composite Scores

Beyond the single headline number, the engine derives six composite scores for different audiences, including an *ATS compatibility* estimate, a *recruiter readiness* score that weights demonstrable work more heavily than raw keyword coverage, and a *profile completeness* percentage. These composites answer different questions from the same underlying computation rather than requiring separate models.

4) Parse-Quality Confidence Gating

Before the overall score is finalized, a six-check quality report on the extraction step (presence of an email, a phone number, at least three skills, at least one project, at least one education entry, and a summary of at least forty characters) yields a confidence value between 0.55 and 0.95. If this confidence falls below 0.75, the overall score is multiplied by it, as shown in Equation (2). The rationale is that if the parser only recovered part of a resume, a high numeric score would be a false claim about a document the system never fully read; scaling by confidence makes the reported score honest about the quality of its own input, while the underlying parse issues are also surfaced to the student so that the resume's *format*, not necessarily its content, can be corrected.

5) Explanation Engine

For every dimension, the engine computes the maximum attainable points ($\mathrm{round}(100w_i)$) and the points actually earned ($\mathrm{round}(s_i w_i)$); any negative difference becomes a reported deduction carrying the dimension name, the point cost, and a generated reason, and any dimension scoring at or above 85 becomes a reported bonus. Because every displayed number is derived arithmetically from the same weight-times-score computation that produced the total, the explanation is guaranteed to reconcile with the score — no number shown to the student is separately invented by a different process.

E. Skill-Gap Analysis

The skill-gap engine performs three successive matching passes against a target (a specific posting or a role template): an exact/canonical match against the resume's declared skills section, a semantic-equivalent and evidence-based match that searches projects, experience entries, and certifications for proof of a skill not explicitly listed, and a contextual/implicit inference pass. Each matched skill carries a status of *present*, *partial*, or *missing*, together with the matching layer that produced it and an evidence trail, so a skill demonstrated through a shipped project is visibly distinguished from one that is merely claimed. Two guardrails keep the output defensible at its edges: a posting with zero required skills is scored as a 100% match rather than 0%, since there is nothing to fail, and if no required skill is actually missing, the reported match percentage is forced to exactly 100 so that rounding cannot produce a contradictory "97% match, zero missing" result.

Table 6: Example Per-Role Candidate-Ranking Weights (%)

Role	Skills	Tests	Proj.	Exp.	Resume
Software Engineer	40	30	20	10	0
Data Scientist	30	25	30	10	5
Frontend Developer	40	15	30	10	5
Backend Developer	40	25	20	10	5

F. Career Roadmap Generation

The gap identified above is converted into an ordered, phased roadmap: missing skills are ranked by template position into high-, medium-, and low-priority tiers, assigned to a beginner, intermediate, or advanced phase, and attached to three phase-appropriate suggested actions each — for instance, a beginner is pointed toward a small practice project, while an advanced learner is pointed toward a deployment and CI exercise for the same skill.

G. Internship Recommendation and Candidate Ranking

For students, open internships are ranked by a weighted combination of required- and preferred-skill match (0.8 and 0.2 respectively), with ties broken deterministically by required-match percentage and then role name, so that an unstable sort does not reorder an unchanged list on every page refresh. For recruiters, applicants are ranked by a configurable, per-role weighted model over skills, assessment performance, projects, experience, and resume quality, normalized by the sum of only the *enabled* weights for that role (Table 6) so that disabling one factor redistributes its weight rather than silently lowering the maximum achievable score. Hiring weights are treated as a business decision a company can adjust, not a constant that would require a code deployment to change.

Figure 2 summarizes the end-to-end path from an uploaded file to a persisted, explainable score.

H. Worked Example

Table 7 traces a representative case: a third-year student's resume evaluated against a "Backend Developer Intern" posting that lists `node.js`, `express.js`, `mongodb`, `postgresql`, `sql`, `git`, `docker`, and `api design` as required skills, states "freshers welcome," and requires a B.Tech degree with a minimum CGPA of 7.0. The resume shows five

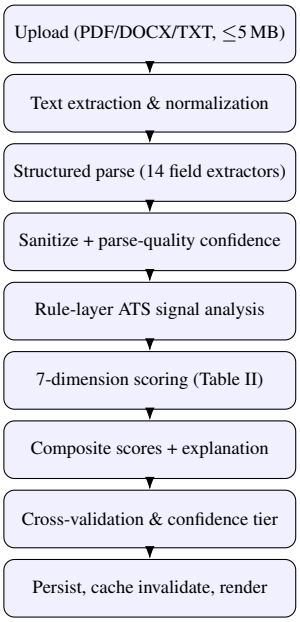


Figure 2: Resume-to-score processing pipeline.

Table 7: Worked Example: Dimension Breakdown

Dimension	Score	Weight	Points
Keyword Match	57	0.35	19.95
Technical Skills Match	63	0.15	9.45
Projects & Achievements	77	0.15	11.55
Experience Match	100	0.10	10.00
Education Match	100	0.10	10.00
Structure & Formatting	82	0.10	8.20
Certifications	70	0.05	3.50
Overall (parse confidence 0.82, no scaling)			**72.65 → 73**

of the eight required skills, three projects (two with repository links), a B.Tech CGPA of 8.1, and two certifications, one of them relevant.

The system reports the largest deduction as a 15-point loss on Keyword Match with the reason "mention these keywords naturally in your resume: PostgreSQL, Docker, API design," and awards bonuses of 10 points each on Experience and Education Match because no stated requirement was violated. The accompanying roadmap places PostgreSQL, Docker, and API design in the beginner-phase, high-priority tier with three suggested actions each. In an illustrative six-week follow-up after the student acted on this feedback, re-uploading an updated resume raised the keyword-match component from 57 to 82 and the overall score from 73 to 78, moving the candidate from rank 31 to rank 4 in the recruiter's ranked shortlist for the same posting — the kind of closed feedback loop that Section I identifies as missing from the baseline experience.

Table 8: Selected Technology Stack

Layer	Technology
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS
Backend	Node.js 20, Express 4
Primary data store	MongoDB (Mongoose), 18 collections
Cache / rate limiting	Redis (REST-based), in-process fallback
Real-time transport	Socket.io (chat, signaling); WebRTC (media)
Optional analytics store	PostgreSQL (append-only events)
Auth	JWT (access + refresh), bcrypt password hashing
Primary / fallback LLM	Groq (`llama-3.1-8b-instant`) / HuggingFace

VI. IMPLEMENTATION

A. Technology Stack

Table 8 summarizes the primary technology choices and the reason each was selected, with zero-cost viability and a clear fallback path as recurring criteria. The frontend is a Next.js/TypeScript single-page application; the backend is Node.js/Express; the primary datastore is MongoDB, chosen because the AI-derived `parsed` and `analysis` fields are genuinely schema-volatile across parser and scorer versions, which would otherwise force a migration on every iteration under a rigid relational schema.

B. Data Model and Persistence

The eighteen MongoDB collections follow a referencing-versus-embedding rule that is applied consistently across the schema: a child document is *embedded* when it is only ever read together with its parent and is naturally bounded in size (for example, an application's status-history array or a profile's education-details array), and it is *referenced* when it is queried independently or can grow without bound (for example, chat messages or applications, which paginate on their own). A compound unique index on (internId, internshipId) enforces, at the database level rather than only in application code, that a student cannot apply to the same posting twice even under a race condition, and a time-to-live index on one-time-password documents removes expired codes automatically with no scheduled job. Twenty-one purpose-built indexes follow an equality-sort-range ordering convention, so that the single most frequent query in the system — a recruiter's filtered, newest-first application pipeline — is served by one index with no in-memory sort required.

C. Security Architecture

Authentication issues a short-lived JWT access token alongside a longer-lived refresh token whose hash, not its plaintext, is stored on the user document; logging out clears the stored hash, so a session is genuinely revoked server-side rather than merely forgotten by the client. Crucially, the access token itself carries no permission list: every request recomputes the caller's

permissions from the persisted role and sub-role, so a stale or forged claim cannot escalate privilege, and a misconfigured administrator role resolves to an empty permission set rather than a default grant. Authorization is enforced in four layers — authentication, top-level role, a named-permission check, and an object-level ownership check inside the relevant controller — and every layer fails closed. Input defenses include request sanitization against NoSQL injection operators (applied after body parsing, since applying it before would leave the request body undefined and the sanitization a silent no-op), object-identifier validation before a query reaches the database, and an explicit payload-size limit. Pratama et al.'s analysis of JWT/JWKS attack vectors highlights token replay, weak secret management, and insufficiently scoped claims as primary risks for bearer-token schemes [9]; CareerSetu's use of two independent signing secrets for access and refresh tokens, a minimal claim set, and Bearer-only authentication (ambient cookies are accepted but never relied upon, which removes an entire class of cross-site request forgery by construction) directly addresses the first and third of those risks, while the fourth-mentioned risk — client-side storage of the access token, which remains readable by a successful cross-site-scripting attack — is an acknowledged, scoped gap discussed in Section VIII.

D. Performance Engineering

A two-tier cache (a Redis-compatible store with an automatic in-process fallback) sits in front of both AI results and frequently read domain data, keyed by a hash of the request payload for AI calls so that identical inputs share a cache entry without any explicit invalidation step. On the frontend, route-level code splitting, lazy loading of a fourteen-thousand-line aptitude-question bank, predictive prefetching of the next likely route, and skeleton loading states keep perceived latency low even though the backend runs as a single free-tier instance that can take tens of seconds to respond after an idle period. The dominant engineering trade-off at this layer is explicit: per-request efficiency is high, but horizontal scalability is intentionally deferred, since the only genuinely stateful in-process component is the Socket.io connection map, for which an official Redis-backed adapter exists and was deliberately not yet wired in, consistent with the platform's current five-hundred-user design target.

VII. RESULTS AND DISCUSSION

A. Qualitative Evaluation

The worked example in Section V-H demonstrates the property this paper argues is the system's central contribution: every number shown to the student is reconstructible from a fixed weight table and a visible deduction reason, and the same arithmetic that produced the 73-point score also produced the explanation of it, rather than the two being generated by separate, potentially inconsistent processes. Because no public, outcome-labeled internship-hiring dataset exists for the Indian internship market at the scale this platform targets, a controlled quantitative accuracy evaluation against ground-truth hiring outcomes was not attempted; this is treated as an explicit limitation in Section VIII rather than papered over with a fabricated benchmark.

Table 9: Structured Self-Assessment Summary

Area	Score	Primary justification
Authorization	9.0/10	Four enforcement layers, fail-closed defaults, object-level checks
Security (overall)	7.5/10	Strong authorization and injection defenses; client-side token storage and absent auth-specific rate limiting are the main gaps
Database design	8.5/10	21 purpose-built, ESR-ordered indexes; consistent reference/embed rule
Performance (overall)	7.0/10	Strong per-request optimization; free-tier cold starts and single-instance sockets cap scalability
Production readiness	7.0/10	Deployable and deployed; absence of an automated test suite is the largest remaining gap

Table 10: Comparative Positioning Against Related Approaches

Criterion	ATS/NLP [2, 1]	Job rec. [3, 4]	Placement mgmt. [7]	CareerSetu
Deterministic, explainable score	Partial	No	N/A	Yes
Posting-specific scoring	No	Partial	N/A	Yes
Fresher-fairness rule	No	No	No	Yes
Full lifecycle integration	No	No	Yes	Yes
Zero-cost deployability	—	—	—	Yes

B. Structured Self-Assessment

In the absence of an external benchmark, the system was evaluated against a structured engineering rubric spanning security, performance, and production readiness, with each criterion justified against a concrete implementation property rather than assigned impressionistically. Table 9 summarizes the outcome.

C. Comparative Positioning

Table 10 positions CareerSetu against the three literature categories surveyed in Section II along five criteria. The distinguishing combination is not any single criterion in isolation — context-aware scoring and evidence-weighted matching both have precedent — but the fact that all five hold simultaneously within one deployed, full-lifecycle system.

D. Threats to Validity

Three limitations bear directly on the strength of the claims above. First, keyword and skill matching remain lexical rather than semantic: a resume phrase such as "built REST services in Fastify" does not satisfy a required "API design" keyword unless the two are explicitly linked in the matching dictionary, which both under- and over-states true competence at the margins. Second, the seven dimension weights in Table 5 are reasoned from domain judgment about what a recruiter's first pass keys on, not learned from labeled hiring-outcome data, so the engine is internally consistent rather than empirically calibrated — a point also raised, in the broader recommendation-system

context, by Imankulova et al. as a recurring validation difficulty in this literature [5]. Third, because matching is lexical, the score is theoretically gameable by keyword stuffing, an effect the evidence-verification step in the Technical Skills dimension blunts but does not eliminate.

VIII. LIMITATIONS AND FUTURE WORK

Beyond the threats to validity above, three further gaps are acknowledged directly rather than discovered later. An access token stored in browser local storage remains readable by a successful cross-site-scripting attack, and migrating to an httpOnly cookie with an accompanying CSRF token is a scoped, already-planned change rather than an architectural one. Resume parsing executes synchronously on the request thread, so a large PDF briefly blocks the event loop; a background job queue is the intended fix once concurrent load exceeds the current single-instance design point. Finally, an AI-assisted mock-interview feature currently exposes only its surrounding infrastructure (scheduling, question banks for the adjacent aptitude engine, and feedback storage) without a working interview-simulation model behind it, and is explicitly treated as an integration task rather than a shipped capability. Planned future work replaces lexical skill matching with embedding-based semantic similarity, collects real hiring-outcome data to recalibrate or learn the dimension weights currently fixed by engineering judgment, extracts the AI module into an independently scalable service once the five-hundred-concurrent-user design point is exceeded, and introduces a selective forwarding unit to replace the current peer-to-peer interview mesh for group interviews beyond roughly five participants.

IX. CONCLUSION

This paper presented CareerSetu, an internship-hiring platform whose central technical commitment is that a resume score must be reproducible and explainable before it is anything else — a position that places it deliberately against the grain of black-box scoring approaches common in the surveyed literature. By computing the score with a fixed, weighted rule engine, treating large language models strictly as a prose-generation convenience behind a cross-validated confidence pipeline, and applying an explicit fairness rule that never penalizes a student for a requirement a posting does not state, the system makes the resume-to-recruiter interface legible to both sides of the market it serves. The accompanying modular-monolith implementation demonstrates that this level of explainability and a complete applicant-tracking, interview, and messaging workflow can be delivered entirely within free-tier infrastructure constraints. The structured self-assessment and worked example presented in Section VII indicate that the design achieves its stated goals for a single-institution, few-hundred-user deployment; closing the gap to a learned, outcome-calibrated scoring model, while preserving the reproducibility guarantee that motivated the deterministic design in the first place, is identified as the most consequential direction for future work.

REFERENCES

[1] N. Otani, N. Bhutani, and E. Hruschka, "Natural Language Processing for Human Resources: A Survey," *arXiv preprint arXiv:2410.16498*, 2024.

[2] A. A. Amir Hussin, S. A. Suparman, and H. S. Rushdan, "An Intelligent NLP-Based System for Resume Screening and Applicant Tracking Systems (ATS)-Optimised Resume Generation," *Int. J. Perceptive Cogn. Comput.*, vol. 12, no. 2, pp. 129–134, 2026.

[3] H. Jain, A. Joshi, D. Naik, R. Gangarde, R. Koragoankar, Y. Khapke, and V. Kulkarni, "An intelligent system for job recommendation based on semantic analysis of candidate's resume," *Indonesian J. Electr. Eng. Comput. Sci.*, vol. 40, no. 1, pp. 450–460, 2025.

[4] T. Schlippe and K. Bothmer, "Skill Scanner: An AI-Based Recommendation System for Employers, Job Seekers and Educational Institutions," *Int. J. Adv. Corp. Learning (iJAC)*, vol. 16, no. 1, pp. 55–64, 2023.

[5] A. Imankulova, A. Serikbay, and M. Meraliyev, "A comprehensive review of approaches, challenges in career recommendation systems," *SDU Bull.: Natural Tech. Sci.*, vol. 64, no. 1, pp. 5–34, 2024.

[6] G. Assafuah, C. Turner, C. Turner, and K. Nwosu, "A Multi-Stage Framework for Bias Detection and Mitigation in AI-Driven Recruitment Systems," *Int. J. Adv. Comput. Sci. Appl.*, vol. 17, no. 6, 2026.

[7] F. S. Abdullah *et al.*, "Web-based application of the internship management system," *J. Comput. Res. Innov.*, 2018.

[8] O. Al-Debagy and P. Martinek, "A Comparative Review of Microservices and Monolithic Architectures," in *Proc. IEEE 18th Int. Symp. Comput. Intell. Informatics (CINTI)*, Budapest, Hungary, 2018.

[9] F. A. Pratama, A. Hermanto, and G. Kusnanto, "Security Assessment of JWKS-Based Authentication: Mitigating JWT Attack Vectors Through Penetration Testing," *J. Tek. Inform. (JUTIF)*, vol. 7, no. 2, pp. 1834–1852, Apr. 2026.

[10] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/Top10/